

Machine Learning Assignment: Text Mining and POS Tagging

Student ID: 2025AIML049

Total Marks: 6

Course: Machine Learning

This notebook demonstrates the implementation of text mining techniques and POS tagging using Hidden Markov Model concepts.

Question 1: Text Mining and Boolean Retrieval (3 Marks)

Objective

In this question, I will implement a complete text retrieval system using the 20 Newsgroups dataset. The task involves:

- Text preprocessing (lowercasing, tokenization, stopwords removal, stemming)
- Creating an inverted index for efficient document retrieval
- Implementing boolean retrieval (AND, OR operations)
- Optimizing queries by sorting terms based on posting list length

Dataset

I will use the 20 Newsgroups Dataset from Scikit-learn with two categories: 'sci.space' and 'rec.sport.hockey'.

```
In [21]: # Import required libraries for text mining
from sklearn.datasets import fetch_20newsgroups
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from nltk.stem import PorterStemmer
import re
from collections import defaultdict

# Fix SSL certificate issue for NLTK downloads (common on macOS)
import ssl
import certifi
ssl._create_default_https_context = ssl._create_unverified_context

# Download required NLTK data packages
# punkt: for word tokenization
# stopwords: for removing common words
nltk.download('punkt')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package punkt to /Users/krishna/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]   /Users/krishna/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
```

```
Out[21]: True
```

Step 1.1: Loading the Dataset

Explanation: I'm loading the 20 Newsgroups dataset with specific parameters:

- `subset='train'` : Using the training portion of the dataset
- `categories=['sci.space', 'rec.sport.hockey']` : Selecting two relevant categories
- `remove=('headers', 'footers', 'quotes')` : Removing metadata to focus on actual content
- Using first 100 documents for this assignment to keep computation manageable

This gives us a focused dataset for demonstrating text mining techniques.

```
In [22]: # Load the 20 Newsgroups dataset
dataset = fetch_20newsgroups(
    subset='train',
    categories=['sci.space', 'rec.sport.hockey'],
    remove=('headers', 'footers', 'quotes')
)

# Take first 100 documents for this assignment
documents = dataset.data[:100]

# Display dataset information
print(f"*60)
print("DATASET LOADING RESULTS")
print(f"*60)
print(f"Number of documents loaded: {len(documents)}")
print(f"Categories: {dataset.target_names}")
print(f"\nFirst document preview (first 200 characters):")
print(f"*60)
```

```
print(documents[0][:200])
print(f"-"*60)

=====
DATASET LOADING RESULTS
=====
Number of documents loaded: 100
Categories: ['rec.sport.hockey', 'sci.space']

First document preview (first 200 characters):
-----
Individual leaders by total points (Final standings)
NOTE: Games played and points per games not accurate !!

Player      Team  GP  G   A  Pts ppg  Prj PIM +/-
-----
M.Lemieux   PIT   59  6
```

Step 1.2: Text Preprocessing

Explanation: Text preprocessing is crucial for effective text mining. I will implement the following steps:

- 1. **Lowercasing:** Convert all text to lowercase to ensure consistency (e.g., "Space" and "space" treated as same word)
- 2. **Tokenization:** Split text into individual words/tokens using NLTK's word_tokenize function
- 3. **Stopword removal:** Remove common words that don't carry much meaning (e.g., 'the', 'is', 'at', 'which')
- 4. **Porter Stemming:** Reduce words to their root form (e.g., 'running' → 'run', 'launched' → 'launch')

This preprocessing helps in reducing the vocabulary size and focusing on meaningful content words.

```
In [23]: # Initialize Porter Stemmer and load stopwords
stemmer = PorterStemmer()
stop_words = set(stopwords.words('english'))

def preprocess_text(text):
    """
    Preprocess text by performing lowercasing, tokenization,
    stopword removal, and Porter stemming.

    Args:
        text (str): Raw text document

    Returns:
        list: List of preprocessed tokens
    """
    # Step 1: Lowercasing - convert to lowercase for consistency
    text = text.lower()

    # Step 2: Tokenization - split into individual words using NLTK
    tokens = word_tokenize(text)

    # Step 3: Stopword removal and filtering
    # - Remove stopwords (common words with little meaning)
    # - Keep only alphabetic tokens (remove numbers, punctuation)
    tokens = [token for token in tokens if token.isalpha() and token not in stop_words]

    # Step 4: Porter Stemming - reduce words to root form
    stemmed_tokens = [stemmer.stem(token) for token in tokens]

    return stemmed_tokens

# Preprocess all documents
print(f"-"*60)
print("TEXT PREPROCESSING")
print(f"-"*60)

processed_documents = []
for i, doc in enumerate(documents):
    processed = preprocess_text(doc)
    processed_documents.append(processed)

# Display preprocessing results for first document
print(f"\nOriginal document (first 100 characters):")
print(f"-"*60)
print(documents[0][:100])
print(f"-"*60)
print(f"\nProcessed tokens (first 20 tokens):")
print(f"-"*60)
print(processed_documents[0][:20])
print(f"-"*60)
print(f"\nNumber of tokens after preprocessing: {len(processed_documents[0])}")
print(f"Number of tokens before preprocessing: {len(word_tokenize(documents[0].lower()))}")
print(f"Reduction in vocabulary: {len(word_tokenize(documents[0].lower())) - len(processed_documents[0])} tokens")
```

TEXT PREPROCESSING

Original document (first 100 characters):

Individual leaders by total points (Final standings)
NOTE: Games played and points per games not

Processed tokens (first 20 tokens):

['individu', 'leader', 'total', 'point', 'final', 'stand', 'note', 'game', 'play', 'point', 'per', 'game', 'accur', 'player', 'team', 'gp', 'g', 'pt', 'ppg', 'prj']

Number of tokens after preprocessing: 1340
Number of tokens before preprocessing: 6867
Reduction in vocabulary: 5527 tokens

Step 1.3: Inverted Index Creation

Explanation: An inverted index is a fundamental data structure in information retrieval. It maps each term to the list of document IDs where that term appears. This allows for efficient document retrieval.

Structure:

- Key: A term (word) from the vocabulary
- Value: List of document IDs where this term appears

Example:

```
{
  "space": [0, 4, 10, 15, 23], # Documents containing 'space'
  "hockey": [1, 3, 8, 12, 20]  # Documents containing 'hockey'
}
```

This structure enables fast boolean operations (AND, OR) on document sets.

```
In [24]: # Create inverted index
# Using defaultdict to automatically handle new keys
inverted_index = defaultdict(list)

print(f"*60)
print("INVERTED INDEX CREATION")
print(f"*60)

# Build the inverted index
for doc_id, tokens in enumerate(processed_documents):
    # Get unique tokens to avoid duplicate document IDs
    # (a term should only appear once per document in the posting list)
    unique_tokens = set(tokens)
    for token in unique_tokens:
        inverted_index[token].append(doc_id)

# Sort the document IDs for each term for efficient operations
for term in inverted_index:
    inverted_index[term].sort()

# Display sample of inverted index
print(f"\nSample of Inverted Index (first 10 terms):")
print(f"*60)
for i, (term, doc_ids) in enumerate(list(inverted_index.items())[:10]):
    print(f'{term}: {doc_ids} (appears in {len(doc_ids)} documents)')
print(f"*60)
print(f"\nTotal unique terms in inverted index: {len(inverted_index)}")
print(f"Total documents indexed: {len(documents)}")

# Show some statistics
avg_posting_length = sum(len(docs) for docs in inverted_index.values()) / len(inverted_index)
print(f"Average posting list length: {avg_posting_length:.2f}")
```

INVERTED INDEX CREATION

Sample of Inverted Index (first 10 terms):

'briseboi': [0] (appears in 1 documents)
'lipuma': [0] (appears in 1 documents)
'baumgartn': [0] (appears in 1 documents)
'reichel': [0] (appears in 1 documents)
'stern': [0] (appears in 1 documents)
'robitail': [0] (appears in 1 documents)
'mclean': [0] (appears in 1 documents)
'hammond': [0] (appears in 1 documents)
'chase': [0, 32] (appears in 2 documents)
'wesley': [0] (appears in 1 documents)

Total unique terms in inverted index: 4264
Total documents indexed: 100
Average posting list length: 2.08

Step 1.4: Boolean Retrieval

Explanation: Boolean retrieval is a classic information retrieval model that uses set operations on posting lists. I will implement:

1. **AND Query:** Returns documents that contain ALL the query terms
- Implemented using set intersection
 - Example: "space AND nasa" → documents containing both 'space' and 'nasa'
2. **OR Query:** Returns documents that contain AT LEAST ONE of the query terms
- Implemented using set union
 - Example: "space OR hockey" → documents containing either 'space' or 'hockey'

These operations are efficient because they work on pre-computed posting lists from the inverted index.

```
In [25]: def boolean_and(query_terms, inverted_index):
        """
        Perform AND query: return documents containing ALL query terms.

        Args:
            query_terms (list): List of query terms
            inverted_index (dict): Inverted index mapping terms to document IDs

        Returns:
            list: Sorted list of document IDs matching all terms
        """
        if not query_terms:
            return []

        # Get posting lists for each term (after stemming)
        posting_lists = []
        for term in query_terms:
            stemmed_term = stemmer.stem(term.lower())
            if stemmed_term in inverted_index:
                posting_lists.append(set(inverted_index[stemmed_term]))
            else:
                # If term not found, return empty set (no documents match)
                return []

        # Perform intersection (AND operation)
        # Documents must appear in ALL posting lists
        if not posting_lists:
            return []

        result = posting_lists[0]
        for posting_list in posting_lists[1:]:
            result = result.intersection(posting_list)

        return sorted(list(result))

def boolean_or(query_terms, inverted_index):
    """
    Perform OR query: return documents containing AT LEAST ONE query term.

    Args:
        query_terms (list): List of query terms
        inverted_index (dict): Inverted index mapping terms to document IDs

    Returns:
        list: Sorted list of document IDs matching any term
    """
    if not query_terms:
        return []

    # Get posting lists for each term (after stemming)
    posting_lists = []
    for term in query_terms:
        stemmed_term = stemmer.stem(term.lower())
        if stemmed_term in inverted_index:
            posting_lists.append(set(inverted_index[stemmed_term]))

    # Perform union (OR operation)
    # Documents can appear in ANY posting list
    if not posting_lists:
        return []

    result = posting_lists[0]
    for posting_list in posting_lists[1:]:
        result = result.union(posting_list)

    return sorted(list(result))

# Test the boolean retrieval functions
print(f"*60)
print("BOOLEAN RETRIEVAL")
print(f"*60)

# Test AND query
query_and = ["space", "nasa"]
print(f"\nAND Query: {' AND '.join(query_and)}")
```

```
print(f"_{60}")
and_results = boolean_and(query_and, inverted_index)
print(f"Matching document IDs: {and_results}")
print(f"Number of documents: {len(and_results)}")

# Test OR query
query_or = ["space", "hockey"]
print(f"\nOR Query: {' OR '.join(query_or)}")
print(f"_{60}")
or_results = boolean_or(query_or, inverted_index)
print(f"Matching document IDs: {or_results}")
print(f"Number of documents: {len(or_results)}")
```

=====

BOOLEAN RETRIEVAL

=====

AND Query: space AND nasa

Matching document IDs: [16, 37]
Number of documents: 2

OR Query: space OR hockey

Matching document IDs: [4, 6, 8, 12, 15, 16, 20, 21, 30, 32, 35, 37, 38, 41, 42, 43, 47, 48, 50, 54, 55, 58, 60, 67, 71, 74, 75, 77, 84, 88, 90, 91]
Number of documents: 32

Step 1.5: Query Optimization

Explanation: Query optimization improves efficiency by processing terms in a strategic order. The key insight is:

- For AND queries: Process terms with **shorter posting lists first**
 - This reduces the size of intermediate result sets
 - Fewer set operations on smaller sets = faster computation
- For OR queries: Process terms with **longer posting lists first** (optional optimization)

Example: If we query "space AND launch AND nasa":

- "launch" might appear in 5 documents
- "nasa" might appear in 20 documents
- "space" might appear in 50 documents

Optimal order: launch → nasa → space (shortest to longest)

This optimization can significantly speed up queries, especially with large datasets.

```
In [26]: def optimized_boolean_and(query_terms, inverted_index):
        """
        Perform optimized AND query by sorting terms by posting list length
        (shortest first for efficient intersection).

        Args:
            query_terms (list): List of query terms
            inverted_index (dict): Inverted index mapping terms to document IDs

        Returns:
            list: Sorted list of document IDs matching all terms
        """
        if not query_terms:
            return []

        # Get posting lists with their lengths
        term_posting_pairs = []
        for term in query_terms:
            stemmed_term = stemmer.stem(term.lower())
            if stemmed_term in inverted_index:
                term_posting_pairs.append((stemmed_term, inverted_index[stemmed_term]))
            else:
                # If term not found, return empty set
                return []

        # Sort by posting list length (ascending - shortest first)
        # This is the key optimization for AND queries
        term_posting_pairs.sort(key=lambda x: len(x[1]))

        print(f"Query terms sorted by posting list length (shortest first):")
        for term, posting_list in term_posting_pairs:
            print(f"  '{term}': {len(posting_list)} documents")

        # Perform intersection in optimized order
        posting_lists = [set(posting_list) for _, posting_list in term_posting_pairs]

        result = posting_lists[0]
        for posting_list in posting_lists[1:]:
            result = result.intersection(posting_list)

        return sorted(list(result))

# Test optimized AND query
```

```
print(f"*60)
print("QUERY OPTIMIZATION")
print(f"*60)

query = ["space", "nasa", "launch"]
print(f"\nOptimized AND Query: {' AND '.join(query)}")
print(f"*60)
optimized_results = optimized_boolean_and(query, inverted_index)
print(f"\nMatching document IDs: {optimized_results}")
print(f"Number of documents: {len(optimized_results)}")

# Compare with non-optimized version to verify correctness
non_optimized_results = boolean_and(query, inverted_index)
print(f"\nNon-optimized results: {non_optimized_results}")
print(f"Results match: {optimized_results == non_optimized_results}")
print(f"\nOptimization maintains correctness while improving efficiency!")
```

```
=====
QUERY OPTIMIZATION
=====

Optimized AND Query: space AND nasa AND launch
-----
Query terms sorted by posting list length (shortest first):
'nasa': 4 documents
'launch': 13 documents
'space': 16 documents

Matching document IDs: [16, 37]
Number of documents: 2

Non-optimized results: [16, 37]
Results match: True

Optimization maintains correctness while improving efficiency!
```

Question 2: POS Tagging using HMM Concepts (3 Marks)

Objective

In this question, I will implement Part-of-Speech (POS) tagging using Hidden Markov Model (HMM) concepts. The task involves:

1. Performing POS tagging on sentences using NLTK
2. Computing transition probabilities between POS tags
3. Computing emission probabilities for words given POS tags
4. Implementing a simplified Viterbi algorithm for POS tag prediction

Dataset

I will use the Treebank Corpus from NLTK, which contains professionally annotated sentences with POS tags.

```
In [27]: # Import required libraries for POS tagging
import nltk
from nltk.corpus import treebank
from nltk import pos_tag, word_tokenize
from collections import defaultdict, Counter
import numpy as np

# Download required NLTK data packages
# treebank: annotated corpus with POS tags
# punkt: for word tokenization
# averaged_perceptron_tagger: for POS tagging
nltk.download('treebank')
nltk.download('punkt')
nltk.download('averaged_perceptron_tagger')
nltk.download('punkt_tab')
nltk.download('averaged_perceptron_tagger_eng')
```

```
[nltk_data] Downloading package treebank to
[nltk_data] /Users/krishna/nltk_data...
[nltk_data] Package treebank is already up-to-date!
[nltk_data] Downloading package punkt to /Users/krishna/nltk_data...
[nltk_data] Package punkt is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data] /Users/krishna/nltk_data...
[nltk_data] Package averaged_perceptron_tagger is already up-to-
[nltk_data] date!
[nltk_data] Downloading package punkt_tab to
[nltk_data] /Users/krishna/nltk_data...
[nltk_data] Package punkt_tab is already up-to-date!
[nltk_data] Downloading package averaged_perceptron_tagger_eng to
[nltk_data] /Users/krishna/nltk_data...
[nltk_data] Package averaged_perceptron_tagger_eng is already up-to-
[nltk_data] date!
```

```
Out[27]: True
```

Step 2.1: Loading the Treebank Corpus

Explanation: The Treebank corpus is a collection of text where each word is annotated with its part-of-speech tag. This is essential for training and evaluating POS tagging systems.

Common POS Tags:

- NN: Noun (singular)
- VB: Verb (base form)
- JJ: Adjective
- DT: Determiner (the, a, an)
- IN: Preposition

I will use the first 200 tagged sentences for this assignment.

```
In [28]: # Load tagged sentences from Treebank corpus
tagged_sentences = treebank.tagged_sents()[0:200]

print(f"*60)
print("TREEBANK CORPUS LOADING")
print(f"*60)
print(f"\nNumber of tagged sentences loaded: {len(tagged_sentences)}")
print(f"\nFirst tagged sentence (first 10 words):")
print(f"*60)
for word, tag in tagged_sentences[0][0:10]:
    print(f"    {word:15s} -> {tag}")
print(f"*60)

# Display some statistics
total_words = sum(len(sentence) for sentence in tagged_sentences)
print(f"\nTotal words in corpus: {total_words}")
print(f"Average words per sentence: {total_words / len(tagged_sentences):.2f}")

=====
TREEBANK CORPUS LOADING
=====

Number of tagged sentences loaded: 200

First tagged sentence (first 10 words):
-----
    Pierre      -> NNP
    Vinken      -> NNP
    ,           -> ,
    61          -> CD
    years       -> NNS
    old         -> JJ
    ,           -> ,
    will        -> MD
    join        -> VB
    the         -> DT
    -----

Total words in corpus: 5111
Average words per sentence: 25.55
```

Step 2.2: POS Tagging

Explanation: POS tagging is the process of marking up a word in a text as corresponding to a particular part of speech. NLTK's `pos_tag` function uses a pre-trained model to assign POS tags to words.

How it works:

- The function takes a list of words as input
- It uses a statistical model (averaged perceptron tagger) to predict the most likely POS tag for each word
- Context is considered (surrounding words influence the tag)

I will demonstrate POS tagging on 5 sentences from the Treebank corpus.

```
In [29]: # Perform POS tagging on 5 sentences
print(f"*60)
print("POS TAGGING DEMONSTRATION")
print(f"*60)

sentences_to_tag = []
for i in range(5):
    # Extract words from tagged sentence
    sentence = [word for word, tag in tagged_sentences[i]]
    sentences_to_tag.append(sentence)

print(f"\nPOS Tagging Results for 5 sentences:\n")
for i, sentence in enumerate(sentences_to_tag):
    # Perform POS tagging using NLTK's pos_tag
    tagged = pos_tag(sentence)
    print(f"Sentence {i+1}:")
    print(f"*60)
    print(f"Words: {' '.join(sentence)}")
    print(f"\nPOS Tags:")
    for word, tag in tagged:
        print(f"    {word:15s} -> {tag}")
```

```
print(f"-"*60)
print()
```

=====

POS TAGGING DEMONSTRATION

=====

POS Tagging Results for 5 sentences:

Sentence 1:

Words: Pierre Vinken , 61 years old , will join the board as a nonexecutive director Nov. 29 .

POS Tags:

Pierre	-> NNP
Vinken	-> NNP
,	-> ,
61	-> CD
years	-> NNS
old	-> JJ
,	-> ,
will	-> MD
join	-> VB
the	-> DT
board	-> NN
as	-> IN
a	-> DT
nonexecutive	-> JJ
director	-> NN
Nov.	-> NNP
29	-> CD
.	-> .

Sentence 2:

Words: Mr. Vinken is chairman of Elsevier N.V. , the Dutch publishing group .

POS Tags:

Mr.	-> NNP
Vinken	-> NNP
is	-> VBZ
chairman	-> NN
of	-> IN
Elsevier	-> NNP
N.V.	-> NNP
,	-> ,
the	-> DT
Dutch	-> NNP
publishing	-> NN
group	-> NN
.	-> .

Sentence 3:

Words: Rudolph Agnew , 55 years old and former chairman of Consolidated Gold Fields PLC , was named *-1 a nonexecutive director of this British industrial conglomerate .

POS Tags:

Rudolph	-> NNP
Agnew	-> NNP
,	-> ,
55	-> CD
years	-> NNS
old	-> JJ
and	-> CC
former	-> JJ
chairman	-> NN
of	-> IN
Consolidated	-> NNP
Gold	-> NNP
Fields	-> NNP
PLC	-> NNP
,	-> ,
was	-> VBD
named	-> VBN
*-1	-> JJ
a	-> DT
nonexecutive	-> JJ
director	-> NN
of	-> IN
this	-> DT
British	-> JJ
industrial	-> JJ
conglomerate	-> NN
.	-> .

Sentence 4:

Words: A form of asbestos once used * * to make Kent cigarette filters has caused a high percentage of cancer deaths among a group of workers exposed * to it more than 30 years ago , researchers reported 0 *T*-1 .

POS Tags:

A	-> DT
form	-> NN
of	-> IN

asbestos	-> NN
once	-> RB
used	-> VBD
*	-> JJ
*	-> NN
to	-> TO
make	-> VB
Kent	-> NNP
cigarette	-> NN
filters	-> NNS
has	-> VBZ
caused	-> VBN
a	-> DT
high	-> JJ
percentage	-> NN
of	-> IN
cancer	-> NN
deaths	-> NNS
among	-> IN
a	-> DT
group	-> NN
of	-> IN
workers	-> NNS
exposed	-> VBN
*	-> RB
to	-> TO
it	-> PRP
more	-> JJR
than	-> IN
30	-> CD
years	-> NNS
ago	-> RB
,	-> ,
researchers	-> NNS
reported	-> VBD
0	-> CD
T-1	-> NN
.	-> .

Sentence 5:

Words: The asbestos fiber , crocidolite , is unusually resilient once it enters the lungs , with even brief exposures to it causing symptoms that *T*-1 show up decades later , researchers said 0 *T*-2 .

POS Tags:	
The	-> DT
asbestos	-> NN
fiber	-> NN
,	-> ,
crocidolite	-> NN
,	-> ,
is	-> VBZ
unusually	-> RB
resilient	-> JJ
once	-> IN
it	-> PRP
enters	-> VBZ
the	-> DT
lungs	-> NNS
,	-> ,
with	-> IN
even	-> RB
brief	-> JJ
exposures	-> NNS
to	-> TO
it	-> PRP
causing	-> VBG
symptoms	-> NNS
that	-> IN
T-1	-> NN
show	-> VBP
up	-> RP
decades	-> NNS
later	-> RB
,	-> ,
researchers	-> NNS
said	-> VBD
0	-> CD
T-2	-> NN
.	-> .

Step 2.3: Transition Probability

Explanation: Transition probability P(tag2 | tag1) represents the probability of transitioning from tag1 to tag2 in a sequence. This is a key component of Hidden Markov Models (HMMs) for POS tagging.

Mathematical Definition:

$$P(\text{tag2} \mid \text{tag1}) = \frac{\text{Count}(\text{tag1} \rightarrow \text{tag2})}{\text{Count}(\text{tag1})}$$

Example:

- $P(NN \rightarrow VB)$ = Probability that a noun is followed by a verb
- $P(DT \rightarrow NN)$ = Probability that a determiner is followed by a noun

These probabilities capture the syntactic structure of English sentences.

In [30]:

```
# Compute transition probabilities
transition_counts = defaultdict(Counter)
tag_counts = Counter()

print(f"*60)
print("TRANSITION PROBABILITY CALCULATION")
print(f"*60)

# Count transitions between tags
for sentence in tagged_sentences:
    for i in range(len(sentence) - 1):
        current_tag = sentence[i][1]
        next_tag = sentence[i + 1][1]
        transition_counts[current_tag][next_tag] += 1
        tag_counts[current_tag] += 1

# Calculate transition probabilities
transition_prob = {}
for tag1 in transition_counts:
    transition_prob[tag1] = {}
    total = tag_counts[tag1]
    for tag2 in transition_counts[tag1]:
        transition_prob[tag1][tag2] = transition_counts[tag1][tag2] / total

# Display sample transition probabilities
print(f"\nSample Transition Probabilities:")
print(f"*60)
print(f"P(NN -> VB): {transition_prob.get('NN', {}).get('VB', 0):.4f}")
print(f"P(VB -> NN): {transition_prob.get('VB', {}).get('NN', 0):.4f}")
print(f"P(DT -> NN): {transition_prob.get('DT', {}).get('NN', 0):.4f}")
print(f"P(NN -> DT): {transition_prob.get('NN', {}).get('DT', 0):.4f}")
print(f"*60)

# Display top transitions for a few tags
print(f"\nTop 5 transitions from tag 'NN' (Noun):")
print(f"*60)
for tag, prob in sorted(transition_prob.get('NN', {}).items(), key=lambda x: x[1], reverse=True)[:5]:
    print(f"    NN -> {tag:5s}: {prob:.4f}")
print(f"*60)

print(f"\nTop 5 transitions from tag 'DT' (Determiner):")
print(f"*60)
for tag, prob in sorted(transition_prob.get('DT', {}).items(), key=lambda x: x[1], reverse=True)[:5]:
    print(f"    DT -> {tag:5s}: {prob:.4f}")
print(f"*60)
```

=====
TRANSITION PROBABILITY CALCULATION
=====

Sample Transition Probabilities:

P(NN -> VB): 0.0000
P(VB -> NN): 0.0367
P(DT -> NN): 0.4387
P(NN -> DT): 0.0030

Top 5 transitions from tag 'NN' (Noun):

NN -> IN : 0.2500
NN -> NN : 0.1265
NN -> , : 0.1113
NN -> NNS : 0.1067
NN -> . : 0.1037

Top 5 transitions from tag 'DT' (Determiner):

DT -> NN : 0.4387
DT -> JJ : 0.2288
DT -> NNP : 0.1108
DT -> NNS : 0.0873
DT -> JJS : 0.0283

Step 2.4: Emission Probability

Explanation: Emission probability $P(\text{word} \mid \text{tag})$ represents the probability of observing a word given a particular POS tag. This is another key component of HMMs.

Mathematical Definition:

$$P(\text{word} \mid \text{tag}) = \frac{\text{Count}(\text{word with tag})}{\text{Count}(\text{tag})}$$

Example:

- $P(\text{"market"} \mid \text{NN})$ = Probability that the word "market" is a noun
- $P(\text{"is"} \mid \text{VBZ})$ = Probability that the word "is" is a verb (3rd person singular)

These probabilities capture which words are commonly associated with which POS tags.

```
In [31]: # Compute emission probabilities
emission_counts = defaultdict(Counter)
tag_word_counts = Counter()

print(f"*60)
print("EMISSION PROBABILITY CALCULATION")
print(f"*60)

# Count word emissions for each tag
for sentence in tagged_sentences:
    for word, tag in sentence:
        emission_counts[tag][word.lower()] += 1
        tag_word_counts[tag] += 1

# Calculate emission probabilities
emission_prob = {}
for tag in emission_counts:
    emission_prob[tag] = {}
    total = tag_word_counts[tag]
    for word in emission_counts[tag]:
        emission_prob[tag][word] = emission_counts[tag][word] / total

# Display sample emission probabilities
print(f"\nSample Emission Probabilities:")
print(f"*60)
print(f"P(word='market' | tag='NN'): {emission_prob.get('NN', {}).get('market', 0):.4f}")
print(f"P(word='is' | tag='VBZ'): {emission_prob.get('VBZ', {}).get('is', 0):.4f}")
print(f"P(word='the' | tag='DT'): {emission_prob.get('DT', {}).get('the', 0):.4f}")
print(f"*60)

# Display top emissions for a few tags
print(f"\nTop 5 words emitted by tag 'NN' (Noun):")
print(f"*60)
for word, prob in sorted(emission_prob.get('NN', {}).items(), key=lambda x: x[1], reverse=True)[:5]:
    print(f"    P('{word:15s}' | NN) = {prob:.4f}")
print(f"*60)

print(f"\nTop 5 words emitted by tag 'VB' (Verb):")
print(f"*60)
for word, prob in sorted(emission_prob.get('VB', {}).items(), key=lambda x: x[1], reverse=True)[:5]:
    print(f"    P('{word:15s}' | VB) = {prob:.4f}")
print(f"*60)
```

=====

EMISSION PROBABILITY CALCULATION

=====

Sample Emission Probabilities:

P(word='market' | tag='NN'): 0.0030

P(word='is' | tag='VBZ'): 0.3864

P(word='the' | tag='DT'): 0.6226

Top 5 words emitted by tag 'NN' (Noun):

P('%' | NN) = 0.0488

P('year' | NN) = 0.0305

P('company' | NN) = 0.0259

P('asbestos' | NN) = 0.0168

P('plant' | NN) = 0.0168

Top 5 words emitted by tag 'VB' (Verb):

P('be' | VB) = 0.1376

P('make' | VB) = 0.0459

P('have' | VB) = 0.0367

P('refund' | VB) = 0.0275

P('complete' | VB) = 0.0183

Step 2.5: Simplified Viterbi Prediction

Explanation: The Viterbi algorithm is a dynamic programming algorithm used to find the most likely sequence of hidden states (POS tags) given a sequence of observations (words).

How it works:

1. **Initialization:** For the first word, calculate probability of each tag based on emission probability
2. **Recursion:** For each subsequent word, calculate the probability of each tag by considering:
 - Transition probability from previous tag
 - Emission probability of current word
 - Best path probability from previous step

3. **Termination:** Find the tag with highest probability for the last word
4. **Backtracking:** Trace back through the best path to get the complete tag sequence

Mathematical Formulation:

$$\delta_t(j) = \max[\delta_{t-1}(i) \times P(\text{tag}_j \mid \text{tag}_i) \times P(\text{word}_t \mid \text{tag}_j)]$$

This algorithm efficiently finds the optimal tag sequence without enumerating all possibilities.

```
In [32]: def simplified_viterbi(sentence, transition_prob, emission_prob, default_tag='NN'):
        """
        Simplified Viterbi algorithm for POS tagging.

        This algorithm finds the most likely sequence of POS tags for a given sentence
        using transition and emission probabilities.

        Args:
            sentence (list): List of words to tag
            transition_prob (dict): Dictionary of transition probabilities P(tag2|tag1)
            emission_prob (dict): Dictionary of emission probabilities P(word|tag)
            default_tag (str): Default tag if word not seen (default: 'NN')

        Returns:
            list: List of predicted POS tags for each word
        """
        if not sentence:
            return []

        # Get all unique tags from training data
        all_tags = list(transition_prob.keys())

        # Initialize Viterbi matrix
        # viterbi[t][tag] = max probability of being at tag at position t
        viterbi = [{ } for _ in range(len(sentence))]

        # Backpointer to reconstruct the path
        backpointer = [{ } for _ in range(len(sentence))]

        # Step 1: Initialization (first word)
        word = sentence[0].lower()
        for tag in all_tags:
            # Emission probability (use small value if word not seen)
            if word in emission_prob.get(tag, {}):
                emit_p = emission_prob[tag][word]
            else:
                emit_p = 1e-10 # Small probability for unknown words

            # Initial probability (assuming equal start probability)
            viterbi[0][tag] = emit_p
            backpointer[0][tag] = None

        # Step 2: Recursion (for remaining words)
        for t in range(1, len(sentence)):
            word = sentence[t].lower()
            for tag in all_tags:
                max_prob = 0
                best_prev_tag = None

                # Find best previous tag
                for prev_tag in all_tags:
                    # Transition probability
                    trans_p = transition_prob.get(prev_tag, {}).get(tag, 1e-10)

                    # Emission probability
                    if word in emission_prob.get(tag, {}):
                        emit_p = emission_prob[tag][word]
                    else:
                        emit_p = 1e-10

                    # Total probability
                    prob = viterbi[t-1][prev_tag] * trans_p * emit_p

                    if prob > max_prob:
                        max_prob = prob
                        best_prev_tag = prev_tag

                viterbi[t][tag] = max_prob
                backpointer[t][tag] = best_prev_tag

        # Step 3: Termination - find best final tag
        best_final_tag = max(viterbi[-1].keys(), key=lambda x: viterbi[-1][x])

        # Step 4: Backtracking to find the best path
        best_path = [best_final_tag]
        for t in range(len(sentence) - 1, 0, -1):
            best_path.insert(0, backpointer[t][best_path[0]])

        return best_path

# Test the Viterbi algorithm on a sample sentence
print(f"={"*60})
```

```
print("VITERBI ALGORITHM - TEST 1")
print(f"*60)

test_sentence = ["the", "market", "is", "open"]
print(f"\nTest sentence: {' '.join(test_sentence)}")
print(f"*60)

# Predict tags using Viterbi
predicted_tags = simplified_viterbi(test_sentence, transition_prob, emission_prob)
print(f"\nViterbi Predicted tags: {predicted_tags}")

# Compare with NLTK's pos_tag (ground truth)
nltk_tags = pos_tag(test_sentence)
print(f"NLTK pos_tag result: {nltk_tags}")

# Display word-tag pairs
print(f"\nWord-Tag pairs (Viterbi):")
print(f"*60)
for word, tag in zip(test_sentence, predicted_tags):
    print(f"    {word:15s} -> {tag}")
print(f"*60)
```

=====
VITERBI ALGORITHM - TEST 1
=====

Test sentence: the market is open

Viterbi Predicted tags: ['DT', 'NN', 'VBZ', 'JJ']
NLTK pos_tag result: [('the', 'DT'), ('market', 'NN'), ('is', 'VBZ'), ('open', 'JJ')]

Word-Tag pairs (Viterbi):

the	-> DT
market	-> NN
is	-> VBZ
open	-> JJ

Step 2.6: Additional Viterbi Test

Explanation: Let me test the Viterbi algorithm on a longer, more complex sentence to evaluate its performance on a realistic example.

```
In [33]: # Test on a longer sentence
print(f"*60)
print("VITERBI ALGORITHM - TEST 2")
print(f"*60)

test_sentence2 = ["the", "company", "announced", "new", "products", "yesterday"]
print(f"\nTest sentence: {' '.join(test_sentence2)}")
print(f"*60)

# Predict tags using Viterbi
predicted_tags2 = simplified_viterbi(test_sentence2, transition_prob, emission_prob)
print(f"\nViterbi Predicted tags: {predicted_tags2}")

# Compare with NLTK's pos_tag (ground truth)
nltk_tags2 = pos_tag(test_sentence2)
print(f"NLTK pos_tag result: {nltk_tags2}")

# Display word-tag pairs
print(f"\nWord-Tag pairs (Viterbi):")
print(f"*60)
for word, tag in zip(test_sentence2, predicted_tags2):
    print(f"    {word:15s} -> {tag}")
print(f"*60)

# Calculate accuracy
nltk_tags_only = [tag for word, tag in nltk_tags2]
matches = sum(1 for p, n in zip(predicted_tags2, nltk_tags_only) if p == n)
accuracy = matches / len(predicted_tags2) * 100
print(f"\nAccuracy compared to NLTK: {accuracy:.2f}% ({matches}/{len(predicted_tags2)} tags match)")
```

Test sentence: the company announced new products yesterday

Viterbi Predicted tags: ['DT', 'NN', 'VBD', 'JJ', 'NNS', 'NN']
NLTK pos_tag result: [('the', 'DT'), ('company', 'NN'), ('announced', 'VBD'), ('new', 'JJ'), ('products', 'NNS'), ('yesterday', 'NN')]

Word-Tag pairs (Viterbi):

the	-> DT
company	-> NN
announced	-> VBD
new	-> JJ
products	-> NNS
yesterday	-> NN

Accuracy compared to NLTK: 100.00% (6/6 tags match)

Summary and Conclusion

Question 1: Text Mining and Boolean Retrieval (3 Marks)

Implemented Components:

1. **Text Preprocessing** (1 Mark)
 - Lowercasing: Converted all text to lowercase for consistency
 - Tokenization: Used NLTK's word_tokenize to split text into words
 - Stopword removal: Removed common words using NLTK's stopwords corpus
 - Porter Stemming: Reduced words to root form using PorterStemmer
 - Result: Significant reduction in vocabulary size while preserving meaning
2. **Inverted Index Creation** (1 Mark)

- Created dictionary mapping terms to document IDs
 - Each term has a sorted list of document IDs where it appears
 - Enables efficient document retrieval operations
 - Total unique terms indexed: [shown in output]
3. **Boolean Retrieval + Query Optimization** (1 Mark)
- Implemented AND query using set intersection
 - Implemented OR query using set union
 - Added query optimization by sorting terms by posting list length
 - Optimization maintains correctness while improving efficiency

Question 2: POS Tagging using HMM Concepts (3 Marks)

Implemented Components:

1. **POS Tagging Implementation** (1 Mark)
 - Used NLTK's pos_tag to tag 5 sentences from Treebank corpus
 - Demonstrated correct POS tag assignment for various words
 - Showed context-aware tagging (same word can have different tags)
2. **Transition & Emission Probability Calculation** (1 Mark)
 - Computed transition probabilities $P(\text{tag2} | \text{tag1})$ from tagged corpus
 - Computed emission probabilities $P(\text{word} | \text{tag})$ from tagged corpus
 - Displayed top transitions and emissions for analysis
 - These probabilities capture syntactic patterns of English
3. **Viterbi Prediction Logic** (1 Mark)
 - Implemented simplified Viterbi algorithm for POS tag prediction
 - Used dynamic programming for efficient computation
 - Tested on multiple sentences and compared with NLTK's pos_tag
 - Algorithm finds most likely tag sequence using HMM principles